

Data Spaces and Federated Data Engineering for Orbital and Space Observations

Laboratory 6: Streaming and Real-Time Federation in a Data Space

Leszek Siwik
Faculty of Space Technologies
AGH University of Krakow

1 Introduction

In Laboratory 5, you built a multi-layer federated data access system based on:

- REST APIs,
- SQL-based federation with DuckDB,
- GraphQL-based semantic integration.

All of these approaches assumed that the data already existed and could be queried as a static dataset.

In real orbital and space observation systems, however, data is rarely static.

Instead, observations are generated continuously by:

- satellites,
- ground stations,
- onboard processing systems,
- mission monitoring pipelines,
- distributed sensing infrastructures.

This leads to the concept of **streaming data**.

In a streaming architecture:

- providers emit events continuously,
- a broker stores and exposes messages,
- consumers retrieve and process them periodically or in near real time,
- federation means combining multiple live streams into one integrated view.

This introduces a fundamental shift: instead of querying a complete dataset, we operate on a **continuously evolving and potentially incomplete data space**.

In this laboratory, you will build a lightweight streaming federation system that simulates real-time data exchange in a data space.

Instead of querying static files, you will:

- publish observation events,
- receive them through a shared broker,
- aggregate them into a unified stream,
- detect missing or delayed providers,
- generate a live report of the federated data space.

The laboratory uses a **Kafka-lite** style approach:

- no heavy enterprise deployment is required,

- providers act as event producers,
- the broker stores and exposes messages,
- consumers retrieve and process federated observations.

In this simplified model, the consumer uses periodic polling instead of true push-based streaming. This makes the system easier to understand and implement, while preserving the key architectural concepts.

During this laboratory, you will learn how to:

- simulate a stream of orbital observation events,
- build simple producers and consumers,
- use a lightweight broker model,
- federate multiple real-time provider streams,
- compute live statistics from streaming data,
- detect missing providers and stream incompleteness,
- generate a time-stamped streaming report.

Important: In the next laboratory, you will move from simulated streaming environments to real-world observation infrastructures, including:

- STAC (SpatioTemporal Asset Catalog),
- Copernicus data services,
- real satellite data discovery and access layers.

This laboratory prepares the conceptual and architectural foundation for that transition.

2 Required Workspace Structure

Use the following workspace structure:

```
oods-system/
  providers/
    satellite_A/
      producer.py
      observations.csv
    satellite_B/
      producer.py
      observations.csv
    ground_station/
      producer.py
      observations.csv
  broker/
    broker.py
  consumers/
    federated_consumer.py
    live_report.py
  contracts/
    event_schema.json
    providers_registry.json
  reports/
```

All commands should be executed from the main project directory, unless explicitly stated otherwise.

3 Streaming Model Used in this Laboratory

In a full enterprise environment, streaming is often implemented using platforms such as Kafka, Pulsar, or Redpanda.

In this laboratory, to keep the system lightweight and transparent, the streaming workflow is simplified.

The architecture consists of:

- **producers** – providers that publish observation events,
- **broker** – a simple service that receives, stores and exposes events,
- **consumer** – a federated component that periodically retrieves events from the broker,
- **reporter** – a component that summarizes the current state of the stream.

Each event represents a single observation and may contain:

- provider,
- timestamp,
- object identifier,
- temperature,
- velocity.

Note that this model simplifies real-world streaming systems:

- events are stored in memory (no persistence),
- there is no partitioning or replication,
- consumers actively poll the broker instead of subscribing to a push stream.

Despite these simplifications, the model captures the key idea of real-time federated data integration in distributed systems.

4 Task 1: Required Tools

Install the required Python packages:

```
pip install fastapi uvicorn pandas requests
```

Verify Python availability:

```
python3 --version
```

This laboratory uses:

- Python 3,
- FastAPI,
- Uvicorn,
- Pandas,
- Requests.

No external stream-processing platform is required in the basic version.

5 Task 2: Event Contract

In streaming systems, providers do not send complete files. Instead, they send messages or events.

Each event should follow a common structure.

Create the file:

```
contracts/event_schema.json
```

with the following content:

```
{
  "required_fields": [
    "provider",
    "timestamp",
    "object_id",
    "temperature",
    "velocity"
  ]
}
```

This contract will define the expected structure of each observation event sent through the stream.

Task

Create an event schema contract that defines the required fields for every observation event.

6 Task 3: Provider Registry

Create the file:

```
contracts/providers_registry.json
```

with the following content:

```
[
  {"name": "satellite_A"},
  {"name": "satellite_B"},
  {"name": "ground_station"}
]
```

This registry will later be used by the consumer to determine which providers are expected in the federation.

Task

Create a registry containing all providers expected in the streaming data space.

7 Task 4: Implementing the Broker

The broker is the central component of the streaming system. Its role is to:

- receive events from producers,
- store them temporarily,
- expose them to consumers.

Create:

```
broker/broker.py
```

with the following example structure:

```
from fastapi import FastAPI
from typing import List

app = FastAPI()
```

```
EVENTS: List[dict] = []

@app.post("/publish")
def publish_event(event: dict):
    EVENTS.append(event)
    return {"status": "accepted", "stored_events": len(EVENTS)}

@app.get("/events")
def get_events():
    return EVENTS
```

This broker:

- accepts events through /publish,
- stores them in memory,
- returns all received events through /events.

Run the broker using:

```
cd broker
uvicorn broker:app --port 7000
```

Task

Create and run a broker service that:

- receives streaming events,
- stores them in memory,
- exposes the current event list for inspection.

Expected Output

After startup, the broker should run on:

```
http://127.0.0.1:7000
```

8 Task 5: Implementing a Provider Producer

Each provider will now act as a producer. Instead of serving static queries, it will read observations from its local CSV file and send them one by one to the broker.

Create:

```
providers/satellite_A/producer.py
```

with the following structure:

```
import pandas as pd
import requests
import time

BROKER_URL = "http://127.0.0.1:7000/publish"
PROVIDER_NAME = "satellite_A"

df = pd.read_csv("observations.csv")

for _, row in df.iterrows():
    event = {
        "provider": PROVIDER_NAME,
```

```

        "timestamp": str(row["timestamp"]),
        "object_id": str(row["object_id"]),
        "temperature": float(row["temperature"]),
        "velocity": float(row["velocity"])
    }

    response = requests.post(BROKER_URL, json=event)
    print("SENT:", event, "STATUS:", response.status_code)
    time.sleep(1)

```

Create analogous producers for:

- satellite_B,
- ground_station.

The `time.sleep(1)` instruction simulates streaming by sending one event at a time.

Task

Create producers that:

- read observations from CSV,
- transform each row into an event,
- send events to the broker one by one,
- simulate a live stream using a delay.

Expected Output

When a producer is executed, it should print messages similar to:

```
SENT: {'provider': 'satellite_A', 'timestamp': '2026-03-01 12:00:00', ...} STATUS: 200
```

9 Task 6: Inspecting the Stream

Once the broker and producers are running, you should be able to inspect all received events.

Open in a browser:

```
http://127.0.0.1:7000/events
```

or use:

```
curl http://127.0.0.1:7000/events
```

This should return the list of events currently received by the broker.

Task

Verify that:

- events arrive at the broker,
- events contain provider identity,
- multiple providers contribute to the stream,
- the event list grows over time.

10 Task 7: Building a Federated Consumer

The next step is to build a consumer that reads events from the broker and creates a federated view.

Create:

```
consumers/federated_consumer.py
```

with the following example structure:

```
import requests
import time

BROKER_EVENTS_URL = "http://127.0.0.1:7000/events"

while True:
    response = requests.get(BROKER_EVENTS_URL)
    events = response.json()

    print("CURRENT NUMBER OF EVENTS:", len(events))

    providers = sorted(set(event["provider"] for event in events))
    objects = sorted(set(event["object_id"] for event in events))

    print("ACTIVE PROVIDERS:", providers)
    print("OBSERVED OBJECTS:", objects)
    print("-" * 40)

    time.sleep(3)
```

This consumer periodically reads the current stream and computes a federated summary.

Task

Create a consumer that:

- polls the broker periodically,
- retrieves all current events,
- determines active providers,
- determines observed objects,
- displays the current stream state.

Expected Output

```
CURRENT NUMBER OF EVENTS: 8
ACTIVE PROVIDERS: ['ground_station', 'satellite_A', 'satellite_B']
OBSERVED OBJECTS: ['OBJ-001', 'OBJ-002', 'OBJ-003']
-----
```

11 Task 8: Real-Time Aggregation

In a federated streaming data space, it is often necessary to compute live statistics.

Extend the consumer so that it also computes:

- number of events per provider,
- total number of distinct objects,
- total number of observations for a selected object,
- most recent event timestamp.

Example logic:

```

per_provider = {}
for event in events:
    provider = event["provider"]
    per_provider[provider] = per_provider.get(provider, 0) + 1

selected_object = "OBJ-003"
selected_count = sum(1 for event in events if event["object_id"] == selected_object)

```

Task

Extend the consumer so that it performs live aggregation for the stream.

Expected Output

```
CURRENT NUMBER OF EVENTS: 10
```

```

PER PROVIDER:
satellite_A: 4
satellite_B: 3
ground_station: 3

```

```

DISTINCT OBJECTS: 5
OBJ-003 OBSERVATIONS: 2

```

12 Task 9: Detecting Missing Providers

In real-time federation, stream incompleteness may result from:

- a provider not sending data,
- a delayed provider,
- network issues,
- failures in the producer.

Using `providers_registry.json`, the consumer can compare:

- expected providers,
- currently active providers.

Example logic:

```

import json

with open("contracts/providers_registry.json", "r") as f:
    expected = [p["name"] for p in json.load(f)]

active = sorted(set(event["provider"] for event in events))
missing = sorted(set(expected) - set(active))

```

This distinction is important: a provider may be temporarily inactive, delayed, or permanently unavailable, and these cases must be interpreted differently in a real-time data space.

Task

Extend the consumer so that it:

- loads expected providers from the registry,
- compares them with active providers,

- detects missing providers,
- marks the current federation as complete or incomplete.

Expected Output

EXPECTED PROVIDERS:

satellite_A
satellite_B
ground_station

ACTIVE PROVIDERS:

satellite_A
satellite_B

MISSING PROVIDERS:

ground_station

COMPLETE: NO

13 Task 10: Event Contract Validation in the Stream

Not every event received in a stream should be trusted automatically.

A consumer should validate whether incoming messages satisfy the event contract.

Load the event contract:

```
contracts/event_schema.json
```

and validate every event against the contract.

Example logic:

```
import json

with open("contracts/event_schema.json", "r") as f:
    contract = json.load(f)

required_fields = contract["required_fields"]

valid_count = 0
invalid_count = 0

for event in events:
    missing = [field for field in required_fields if field not in event]

    if missing:
        print("INVALID EVENT:", event)
        print("MISSING FIELDS:", missing)
        invalid_count += 1
    else:
        valid_count += 1

print("VALID EVENTS:", valid_count)
print("INVALID EVENTS:", invalid_count)
```

Task

Extend the consumer so that it:

- validates all events against the contract,
- detects malformed events,
- reports invalid messages,
- counts valid and invalid events.

Expected Output

```
VALID EVENTS: 14
INVALID EVENTS: 1

INVALID EVENT DETECTED:
Missing fields: velocity
```

14 Task 11: Generating a Live Streaming Report

Create:

```
consumers/live_report.py
```

This script should:

- fetch the current event stream from the broker,
- validate events,
- compute provider statistics,
- compute object-level statistics,
- detect missing providers,
- generate a structured report file.

The report should be saved as:

```
reports/stream_report_YYYYMMDD_HHMMSS.txt
```

Report Requirements

The report must include:

- generation time,
- total number of received events,
- valid and invalid event counts,
- active providers,
- missing providers,
- observations per provider,
- distinct object count,
- statistics for a selected object,
- current completeness status of the stream federation.

Expected Output

```
REAL-TIME FEDERATION REPORT
-----
Generated at: 2026-04-20 12:10:00

[STREAM STATUS]
Total events: 15
```

```
Valid events: 15
Invalid events: 0
```

[PROVIDERS]

Active providers:

- satellite_A
- satellite_B
- ground_station

Missing providers:

- none

[PER-PROVIDER COUNTS]

satellite_A: 6

satellite_B: 5

ground_station: 4

[OBJECT STATISTICS]

Distinct objects: 6

OBJ-003 observations: 3

[FEDERATION STATUS]

COMPLETE: YES

15 Practice Tasks

The following tasks simulate more realistic real-time engineering problems in a streaming data space.

Each task should be implemented in Python. **Complete at least two selected practice tasks.**

Task P1: Delayed Provider Detection

Modify one producer so that it starts sending events with a delay of several seconds.

The consumer should:

- detect that the provider is temporarily missing,
- later detect that it became active,
- observe how federation completeness changes over time.

Task P2: Duplicate Event Detection

Create logic that identifies whether the same event has been received more than once.

Assume that a duplicate is defined as an event with the same:

- provider,
- timestamp,
- object_id.

Expected output may be:

```
DUPLICATE EVENT DETECTED:
provider=satellite_A
timestamp=2026-03-01 12:10:00
object_id=OBJ-003
```

Task P3: High-Frequency Provider Simulation

Modify one producer so that it sends events faster than the others, for example every 0.2 seconds instead of every 1 second.

Then observe:

- which provider dominates the stream,
- whether one provider skews the real-time picture,
- whether the federated stream appears balanced or biased.

Task P4: Object Alert Detector

Extend the consumer so that it prints an alert whenever an event for a selected object appears.

Example:

```
ALERT: OBJ-003 observed by satellite_B at 2026-03-01 12:12:00
```

Task P5: Threshold Alert

Extend the consumer so that it prints an alert when:

- temperature exceeds a selected threshold,
- velocity exceeds a selected threshold.

Example:

```
WARNING:  
Provider: ground_station  
Object: OBJ-004  
Temperature: 28.4
```

Task P6: Sliding Window Statistics

Instead of analyzing all events ever received, modify the consumer so that it analyzes only the most recent N events, for example:

- last 5 events,
- last 10 events.

This simulates a stream-processing window.

Task P7: Data Space Health Indicator for Streaming

Create a script:

```
consumers/stream_health.py
```

The script should combine:

- provider completeness,
- event validity,
- duplicate rate,
- delayed providers,
- selected alert conditions.

Classify the streaming data space as:

- GOOD,
- WARNING,
- CRITICAL.

Expected output may be:

```
STREAM HEALTH: WARNING
Reason: delayed provider and malformed events detected
```

Task P8: Kafka-Style Reflection Task

Write a short note in:

```
reports/streaming_reflection.txt
```

Answer:

- how the implemented broker differs from Kafka,
- which Kafka features are missing,
- what would need to be added to make the architecture closer to a real streaming platform.

16 Submission

Submit your work using both a version control repository and an archive uploaded to the UPEL platform.

Both submissions are required for the laboratory to be considered complete.

Repository Submission

Your repository must include:

- all producer scripts in the `providers` directory,
- the broker implementation in the `broker` directory,
- all consumer scripts in the `consumers` directory,
- the event contract and registry files in the `contracts` directory,
- generated reports in the `reports` directory.

After completing the laboratory:

- commit your changes,
- push them to your repository,
- create a **Merge Request** (GitLab) or **Pull Request** (GitHub).

The request must clearly indicate that it contains the solution for:

```
Laboratory 6: Streaming and Real-Time Federation in a Data Space
```

UPEL Submission

Upload an archive containing the complete workspace:

```
oods-system/
  providers/
  broker/
  consumers/
  contracts/
  reports/
```

The archive must be named:

```
lab06_oods_<surname>.tar.gz
```

Report File Requirement

The submitted report must contain:

- stream statistics,
- provider completeness analysis,
- event validation summary,
- object-level summary,
- current state of federated real-time integration.

Final Note

This laboratory introduces an important shift: from querying existing data to processing live federated streams.

This is one of the key ideas behind modern distributed data systems:

- providers continuously emit new data,
- integration happens in real time,
- federation is dynamic rather than static,
- engineering decisions must often be made on incomplete, delayed or evolving streams.

In contrast to previous laboratories, where completeness could be assumed, here you must explicitly reason about:

- missing providers,
- delayed data,
- partial observability of the system.

Understanding this transition is essential for building realistic data spaces and federated data engineering solutions for orbital and space observation systems.

This knowledge will be directly used in the next laboratory, where you will work with real-world satellite data ecosystems based on STAC and Copernicus platforms.